



Parcial 2 - Arquitectura de computadoras 2025

Nombre: _____

DNI: _____

Ejercicio 1

Analizar el rendimiento de un sistema que utiliza un procesador de 8 bits con un bus de direcciones de 16 bits y memoria direccionable de a byte. Se están comparando dos configuraciones de caché (Caché 1 y Caché 2), ambas con una capacidad total de almacenamiento de datos de 64 bytes y un tamaño de bloque de 16 bytes.

Caché 1: Es de Correspondencia Directa.

Caché 2: Es Asociativa por Conjuntos de 2 Vías (2-Way) y utiliza una política de sustitución LRU (Least Recently Used).

Ambas cachés están inicialmente vacías (todas las líneas marcadas como inválidas). Se ejecuta la siguiente secuencia de accesos a memoria (direcciones en hexadecimal):

0x0A10, 0x0A52, 0x0A18, 0x0A5C, 0x0A10, 0x0B10

Se Pide:

- a) Para ambas configuraciones (Caché 1 y Caché 2), determine cuántos bits hay en cada campo del formato de la dirección:

	Tag	Index/Set	Word	OffsetByte
CACHE 1				
CACHE 2				

- b) Simule la secuencia de accesos SOLO en CACHE 2. Para cada acceso, indique si fue Hit o Miss; el número de línea o conjunto involucrado; y su estado final (contenidos del Tag y Datos) después del acceso. Para el campo de Datos utilice el formato de dirección de la primera y de la última palabra almacenada en la línea (Ej. M[0x0108] .. M[0x0100])

Acc	Address	Set	H/M	Tag (hex) V0	Datos V0	Tag (hex) V1	Datos V1
1	0x0A10				M[] ... M[]		M[] ... M[]
2	0x0A52				M[] ... M[]		M[] ... M[]
3	0x0A18				M[] ... M[]		M[] ... M[]
4	0x0A5C				M[] ... M[]		M[] ... M[]
5	0x0A10				M[] ... M[]		M[] ... M[]
6	0x0B10				M[] ... M[]		M[] ... M[]

Parcial 1 - Arquitectura de computadoras 2025

Nombre: _____

c) Calcule la Tasa de Misses (Miss Rate) para cada caché

CACHE 1:%

CACHE 2:%

Ejercicio 2

Un compilador (aka "estudiante") debe optimizar un bucle crítico para un procesador LEGv8 con arquitectura static 2-issue. El hardware tiene un pipeline de 5 etapas (IF, ID, EX, MEM, WB) y ejecuta un paquete de dos instrucciones por ciclo, con las siguientes restricciones:

- **Slot 1:** Ejecuta operaciones ALU (ADD, SUB, ADDI, ORR) o Saltos (B, CBNZ).
- **Slot 2:** Ejecuta operaciones de Carga (LDUR) o Almacenamiento (STUR).

El procesador es estático, tiene todos los caminos de forwarding implementados, y NO genera stalls por hazards de datos. El compilador (es decir, usted) es responsable de insertar NOPs o reordenar el código para evitar conflictos.

Considere el código del **Ejercicio 4**. Aplicando **solo** la técnica de **Reordenamiento de Código** conforme los issue-packets a fin de MINIMIZAR los ciclos de clock necesarios para ejecutar una iteración del lazo.

Issue	Slot 1	Slot 2
1	>	>
2	.	.
3	.	.
4	> .	> .
5	>	.
6	>	>
7	>	>
8	>	>
9	>	>

Ejercicio 3

Considerar que un procesador que cuenta con un predictor global de 8 bits ejecuta el siguiente código en C y compilado como se muestra a la derecha:

```
do {
    // X0 <- i
    if (i % 2 == 0){ // Si i es par, no salto
        // ...
    }
    if (i % 2 != 0){ // Si i es impar, no salto
        // ...
    }
    i++;
} while (i < 3)
```

```
...
0x10  LOOP_FOR: ...
...
0x24  B.NE    FIN_IF_1
...
0x30  FIN_IF_1: ...
0x38  B.NE    FIN_IF_2
...
0x44  FIN_IF_2: ...
0x50  CBNZ   LOOP_FOR
```

Parcial 1 - Arquitectura de computadoras 2025

Nombre: _____

Completar la siguiente tabla con todas las posiciones de la PHT que se modifican, considerar que todas las posiciones de la tabla están inicializadas en "01" y el GR=0x61

Dirección	Contenido

Ejercicio 4

Considerando un microprocesador out-of-order execution implementado mediante el algoritmo de Tomasulo que cuenta con las características del hardware descritas en la tabla y que ejecuta el siguiente código en assembler, muestre el contenido de las tablas de estado de las Reservation stations y registros (en la próxima página) para el 3er clock de ejecución (inclusive y considerando que se hace fetch en el clk 0). Considerar que los LDUR y los STUR se almacenan en los mismos RS (3 RS en total para los loads y store).

```

1>      ADD X5, XZR, XZR
2>  L1:  LDUR X8, [X0, #0]
3>      LDUR X7, [X0, #0]
4>      ADD X6, X8, X7
5>      SUB X5, X8, X7
6>      STUR X6, [X0, #0]
7>      STUR X5, [X0, #0]
8>      ADDI X0, X0, #8
9>      SUBIS X2, X2, #1
10>     B.LT L1
    
```

Hardware
<i>El salto se predice Taken correctamente</i>
Issue = 4 instrucciones
Load y Store = 3 RS / 1 clk
ALU entera = 4 RS / 1 clk
ALU punto flotante = 4 RS / 1 clk
Multiplicación entera = 2 RS / 2 clk

Name	Reservation stations						
	Busy	Op	Vj	Vk	Qj	Qk	A
Load - Store 1							
Load - Store 2	☐						
Load - Store 3	☐						
alu int 1	☒						
alu int 2							-
alu int 3	☐						-
alu int 4	☐						
alu fp 1	☐						
alu fp 2	☐						
alu fp 3	☐						
alu fp 4	☐						
mult fp 1	☐						
mult fp 2	☐						
Branch	☐						

[illegible]